IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Atul Kumar

04/28/2025

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

Goal
•Estimate the price of each SpaceX launch.
•Predict whether first-stage boosters would be reused.
•Build an interactive dashboard to visualize SpaceX launch data.
Summary of Methodologies
•Collected SpaceX launch data through API access and web scraping.
•Cleaned, filtered, and processed data with Pandas and SQL.
•Conducted exploratory data analysis (EDA) and created visualizations.
•Built an interactive dashboard using Folium and Plotly Dash.
•Trained and evaluated machine learning models (Logistic Regression, SVM, KNN, Decision Tree) with GridSearchCV tuning.
Summary of All Results
•Launch success rates improved over time, especially for heavier payloads and newer booster versions.
•KSC LC-39A had the highest launch success rate (~77%).
•Decision Tree Classifier achieved the best performance, with 88% accuracy in predicting booster reuse.
•The dashboard allows dynamic exploration of launch success based on site, payload, and booster version, providing actionable insights.

Introduction

- Project background and context
- The commercial space industry is growing, with SpaceX leading through innovation and cost efficiency. Its key achievements include:
  - Manned space missions
  - Starlink satellite internet
  - ISS supply missions
- A major factor in SpaceX's success is its ability to reuse the Falcon 9's first stage, reducing launch costs to ~$62M (vs. $165M for others).
- In this project, as a data scientist at Space Y, a new competitor to SpaceX, tasked with analyzing launch data and building dashboards.

Introduction

- Problems I want to find answers:
- As a Space Y data scientist, goals are to:
  - Estimate launch costs
  - Predict first-stage reuse using machine learning
  - Identify mission factors affecting reuse (e.g., payload, orbit)
  - Build interactive dashboards to visualize insights
  - Use data science, not rocket science, to inform decisions

Section 1

Methodology

Methodology

Executive Summary

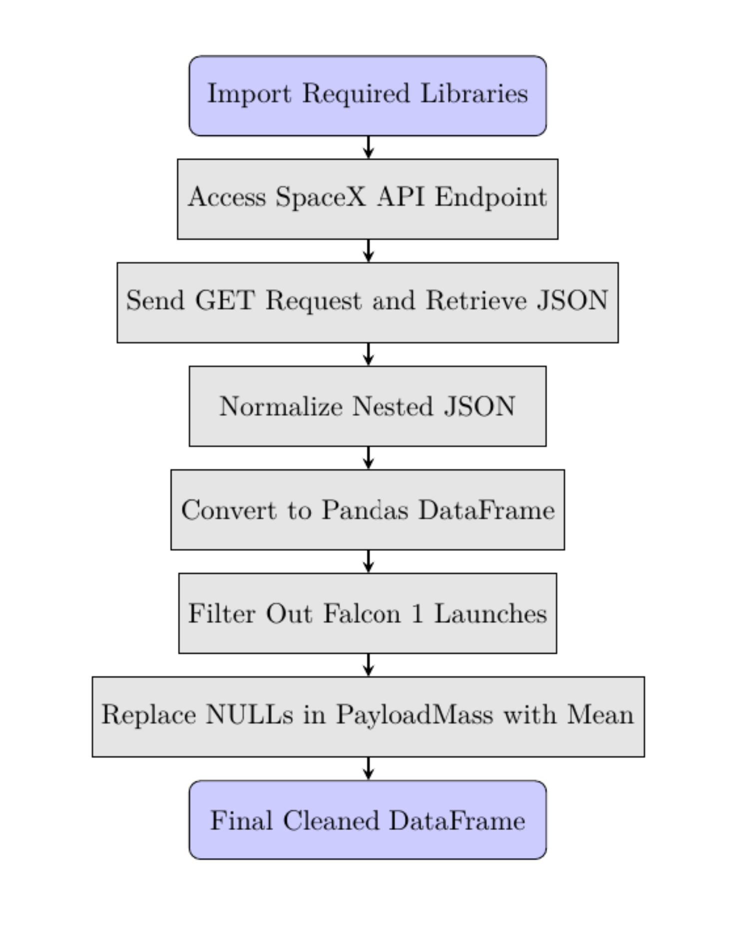
- Data collection methodology:
  - SpaceX launch data was collected via the SpaceX API and Wikipedia web scraping.
- Perform data wrangling
  - Dropped irrelevant columns and cleaned data.
  - Merged datasets, normalized features, and prepared a final clean dataset for analysis and modeling.
  - Applied one-hot encoding to categorical variables.
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models

Data Collection – SpaceX API

- How data sets were collected.
  - Collected SpaceX launch data using the SpaceX REST API (/launches/past)
  - Used requests.get() to retrieve JSON data and json_normalize() to flatten it
  - Extracted key launch details: rocket, payload, launch specs, landing outcome
  - Performed web scraping on Wikipedia using BeautifulSoup
  - Parsed Falcon 9 launch records into a structured Pandas DataFrame
  - Filtered out Falcon 1 launch data to focus only on Falcon 9
  - Handled missing values:
  - Replaced NULLs in PayloadMass with its column mean
  - Left NULLs in LandingPad for one-hot encoding later
  - Final result: a clean, structured dataset ready for visualization and machine learning

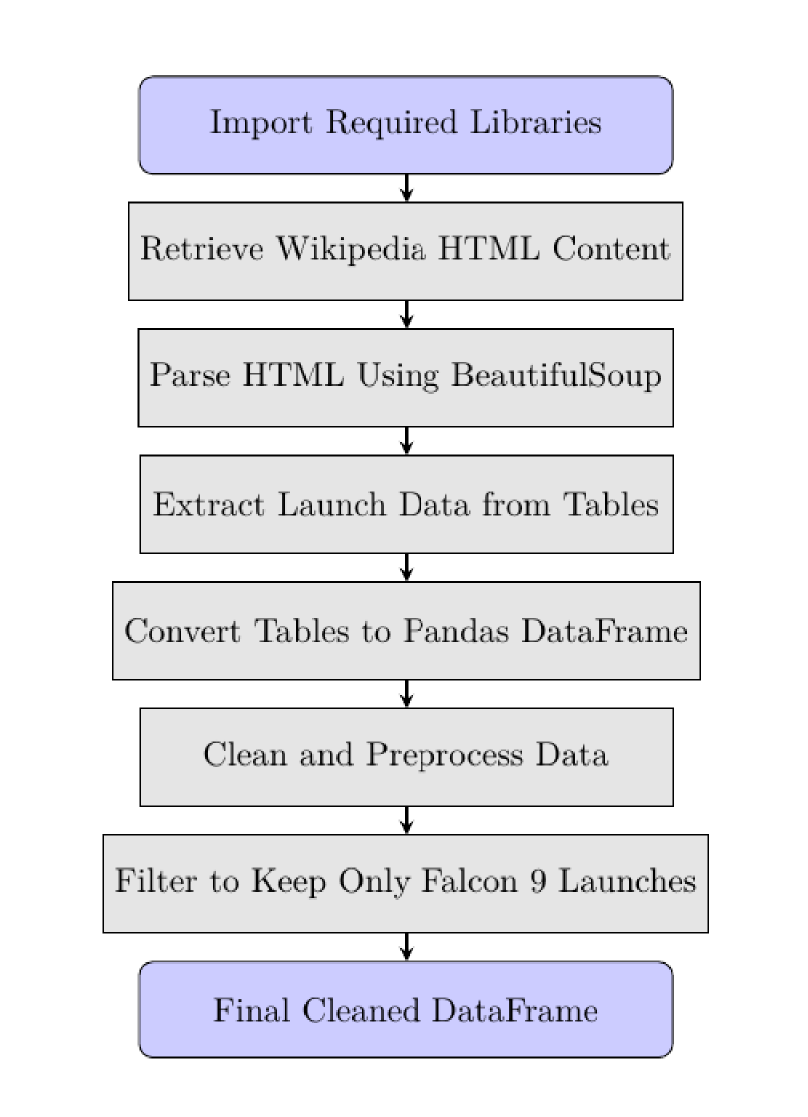
- Github URL: https://github.com/dharlabwustl/datasciencetools/blob/master/jupyter-labs-spacex-data-collection-api.ipynb

Place
calls
here



Data Collection - Scraping

- Web scrapping:
  - Import required libraries
  - Retrieve Wikipedia HTML content using HTTP request
  - Parse HTML content using BeautifulSoup
  - Extract launch data from HTML tables
  - Convert extracted tables to Pandas DataFrame
  - Clean and preprocess table data (handle NULLs, etc.)
  - Filter data to include only Falcon 9 launches
  - Final cleaned DataFrame ready for analysis

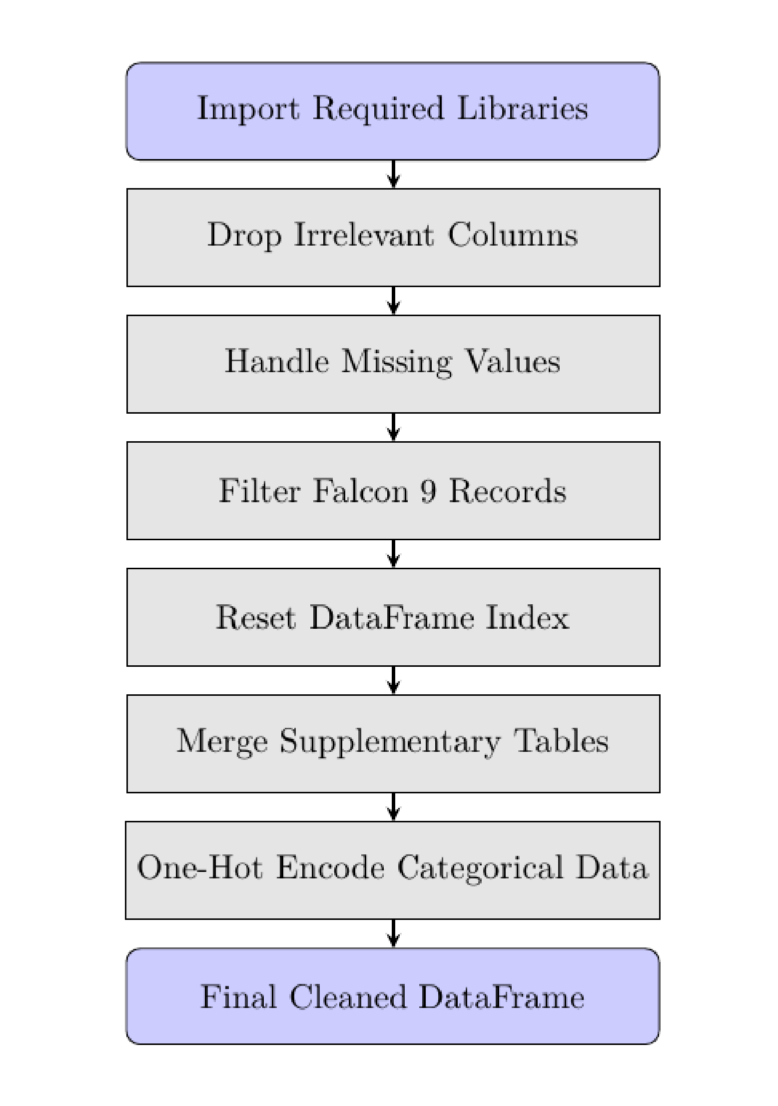


- Github URL: https://github.com/dharlabwustl/datascie ncetools/blob/master/jupyter-labs-webscraping.ipynb

Data Wrangling

- Data Wrangling Steps
    - Import required libraries (pandas, numpy, etc.)
    - Drop irrelevant or redundant columns
    - Handle missing values (e.g., drop rows or fill with statistics)
    - Filter records to include only Falcon 9 launches
    - Reset DataFrame index after filtering
    - Merge supplementary tables like booster info or launch site data
    - Apply One-Hot Encoding to categorical columns for modeling
    - Final cleaned dataset is ready for analysis and visualization

    Git hub link: https://github.com/dharlabwustl/datasciencetools/blob/master/labs-jupyter-spacex-Data%20wrangling.ipynb

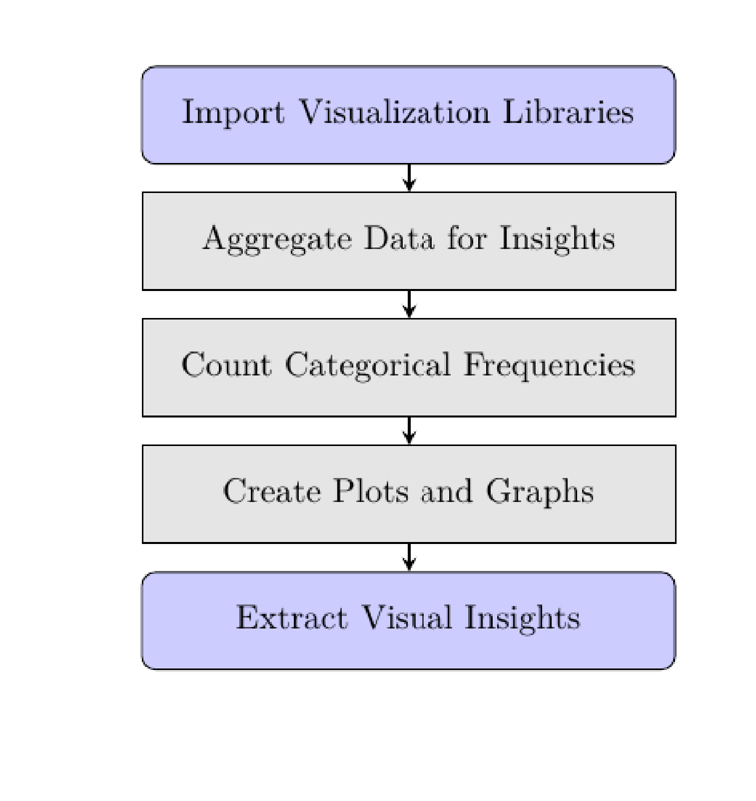


EDA with Data Visualization

EDA and Visualization Steps

- Import visualization libraries (matplotlib, seaborn, plotly)
- Aggregate data using groupby() and statistical functions (e.g., mean)
- Count frequencies of categorical values (e.g., value_counts())
- Create a variety of plots (bar, pie, scatter, etc.) to explore relationships and distributions
- Final output: Visual insights into SpaceX launch data distributions and trends

- Github: https://github.com/dharlabwustl/datasciencetools/blob/master/edadataviz.ipynb

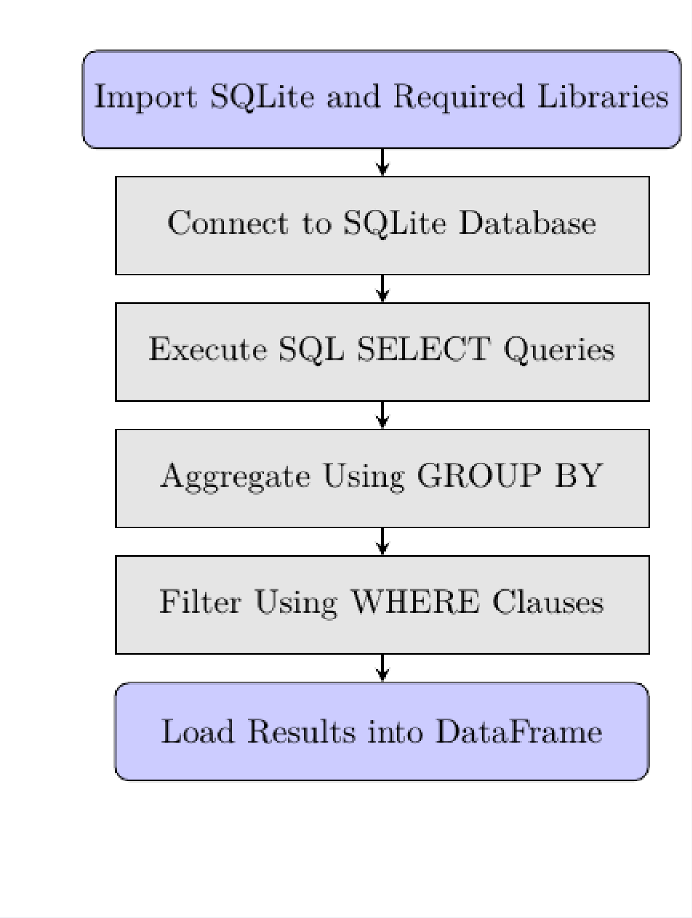


EDA with SQL

SQL EDA Steps

- Import SQLite and required libraries (sqlite3, pandas)
- Connect to the SQLite database
- Execute SQL SELECT queries to retrieve relevant data
- Use GROUP BY clauses to aggregate data (e.g., success rates per orbit)
- Filter records with WHERE clauses (e.g., by launch year or site)
- Load SQL query results into Pandas DataFrames for further analysis

- Github link: https://github.com/dharlabwustl/datasciencetools/blob/master/jupyter-labs-eda-sql-coursera_sqllite.ipynb



Build an Interactive Map with Folium

- Created a base map
  - A Folium map was initialized centered on NASA Johnson Space Center.
  - Later, another map was initialized centered on the average location of all launch sites.
  - This provided a starting point to place markers, circles, and other objects to visualize the data.
- Added Circles for Launch Sites
  - A folium.Circle was added at each launch site location.
  - Each circle had a radius of 1000 meters, was colored orange, and showed the Launch Site name when clicked.
  - Circles highlighted the area around each launch site and made the map easier to understand.
- Added Text Markers (DivIcon) for Launch SitesA folium.
  - Marker with a custom text label (using DivIcon) was placed at each launch site.
  - The text directly displayed the Launch Site name on the map without needing to click.
  - Text markers improved the visibility and readability of the site names.

Build an Interactive Map with Folium

- Created a Marker Cluster for Launch Sites
  - A MarkerCluster was created to group many launch site markers together
  - Each marker had a rocket icon and color based on a property (marker_color), and showed the Launch Site name in a popup.
  - Clustering helped organize markers and avoided clutter when many launch sites were close together.
- Added Distance Markers to Coastline, Highway, and City
  - Distances from a launch site to the nearest coastline, highway, and city were calculated.folium.
  - Marker objects were placed at each nearest point with text labels showing the distance (e.g., "12.34 KM").
  - These distance markers provided additional information about the surrounding geography of each launch site.

Build an Interactive Map with Folium

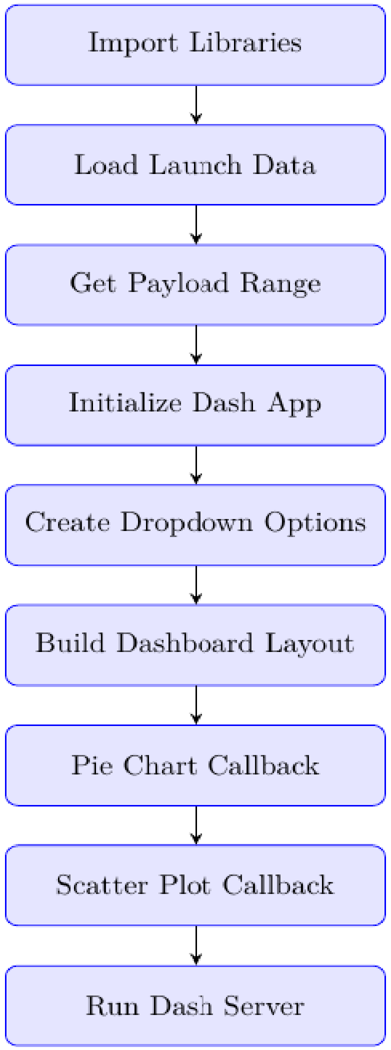
- Drew Lines from Launch Sites to Coastline, Highway, and Cityfolium.
  - PolyLine objects were drawn from each launch site to its nearest coastline, city, and highway.
  - The lines made it easier to visually understand the distance and direction to important nearby locations.
- Added Mouse Position Tool
  - The MousePosition plugin was imported and set up.
  - This tool allowed users to see the exact latitude and longitude while moving the mouse over the map.
  - It enhanced the interactive experience when exploring the map.

https://github.com/dharlabwustl/datasciencetools/blob/master/lab_jupyter_launch_site_location%20(3).ipynb

Build a Dashboard with Plotly Dash

Steps with Purpose

•Import libraries: For data handling (Pandas), web app (Dash), and plots (Plotly)
•Load launch data: Read spacex_launch_dash.csv for interactive analysis
•Get payload range: Define slider bounds (min/max payload)
•Initialize Dash app: Set up the dashboard framework
•Create dropdown options: Enable filtering by launch site
•Build dashboard layout: Add title, dropdown, pie chart, slider, and scatter plot
•Pie chart callback: Show success distribution (all sites or per site)
•Scatter plot callback: Show payload vs. success; colored by booster version
•Run Dash server: Launch the app locally
•GitHub Link: https://github.com/dharlabwustl/datasciencetools/blob/master/spacex-dash-app.py



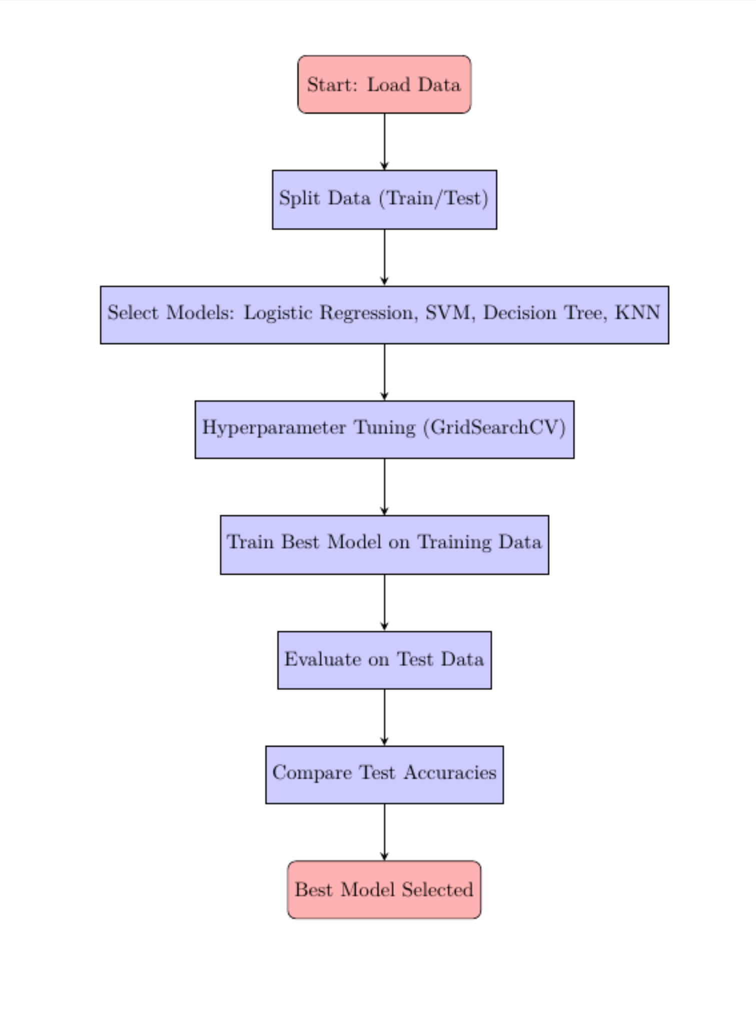
Predictive Analysis (Classification)

- Data Splitting:
  - The dataset was split into training (80%) and testing (20%) sets using train_test_split().
- Model Selection:
  - Four different classification models were chosen for testing:
    - Logistic Regression
    - Support Vector Machine (SVM)
    - Decision Tree
    - K-Nearest Neighbors (KNN)
- Hyperparameter Tuning:
  - Each model was tuned using GridSearchCV with 10-fold cross-validation.
  - The grid search tried different values of parameters like C (for regularization strength) and penalty (l2 ridge penalty).
- Training:
  - All models were trained on the training set using the best parameters found by GridSearchCV.

Predictive Analysis (Classification)

- Evaluation on Training Set:
  - After tuning, the best accuracy on the training data (cross-validation accuracy) was printed for each model.
- Evaluation on Testing Set:
  - Each best-tuned model was tested on the unseen test data.
  - Test accuracy scores were printed for comparison.
- Confusion Matrix:
  - For each model, a confusion matrix was plotted to better understand the prediction performance (true positives, false positives, etc.).
- Best Model Selection:
  - After comparing all models, the one with the highest test accuracy was selected as the best performing classification model.
- Github LINK:https://github.com/dharlabwustl/datasciencetools/blob/master/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

Predictive Analysis (Classification)



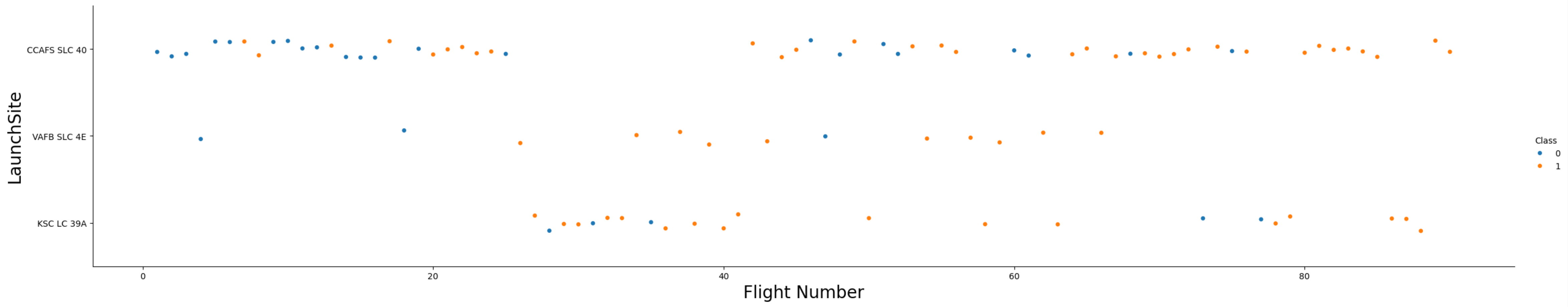
Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

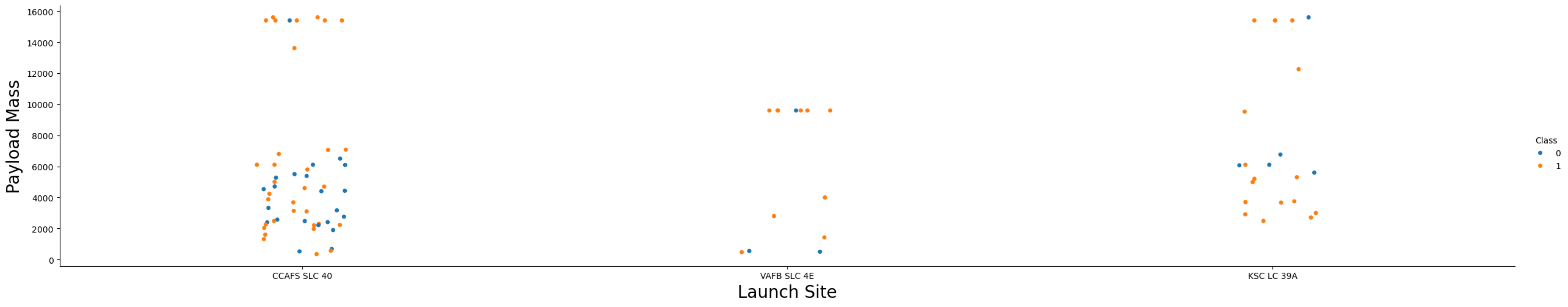


Flight Number vs. Launch Site

Observations from the Plot:

1. Early Flights (Low Flight Numbers):
   - More blue dots (Class 0) are seen for early flights, meaning early launches had more failures.
2. Later Flights (Higher Flight Numbers):
   - More orange dots (Class 1), meaning success rate improved over time.
   - This shows that as SpaceX gained experience, the success rate increased.
3. Launch Sites Differences:
   - CCAFS SLC 40 (Cape Canaveral) had a lot of early activity (more crowded points).
   - KSC LC 39A and VAFB SLC 4E have fewer points overall, and they seem to appear later (higher flight numbers).
   - This suggests that SpaceX started using more launch sites after gaining more flight experience.
4. Pattern across Sites:
   - Across all launch sites, later flights (higher Flight Numbers) show more successes (more orange), indicating consistent improvement across sites.

Payload vs. Launch Site

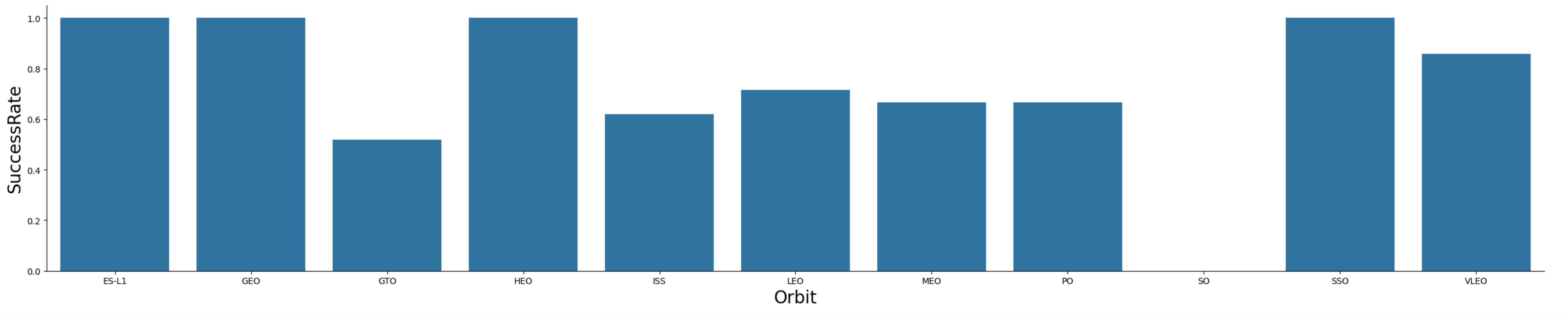


Payload vs. Launch Site

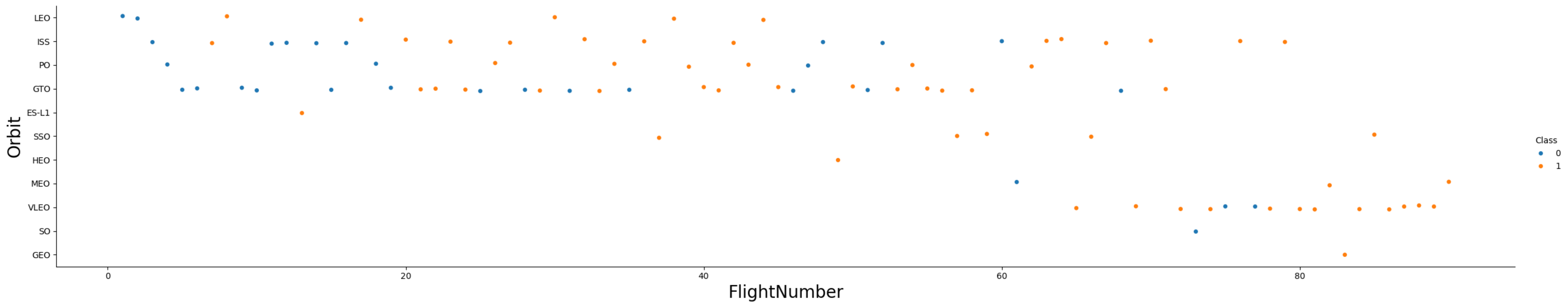
Observations from the Plot:

1. Payload Mass Range at CCAFS SLC 40:
    1. Wide range of payloads from 0 to over 16,000 kg.
    2. Both successes and failures are scattered throughout, but:
        1. Heavy payloads (near 16,000 kg) mostly succeeded (more orange).
        2. Lower payloads had a mixed success/failure outcome.
2. Payload Mass Range at VAFB SLC 4E:
    1. Generally smaller payloads compared to CCAFS.
    2. Few launches overall.
    3. Successes slightly more frequent than failures.
3. Payload Mass Range at KSC LC 39A:
    1. Payload mass distribution similar to CCAFS:
        1. Cluster around moderate to high payloads (4000-16,000 kg).
    2. Heavier payload launches had more successful outcomes (more orange at the top).
4. Overall Trend:
    1. Across all sites, heavier payloads tend to have more successful outcomes.
    2. Light payload launches show more variability (some failed, some succeeded).

Success Rate vs. Orbit Type



Success Rate vs. Orbit Type



Success Rate vs. Orbit Type

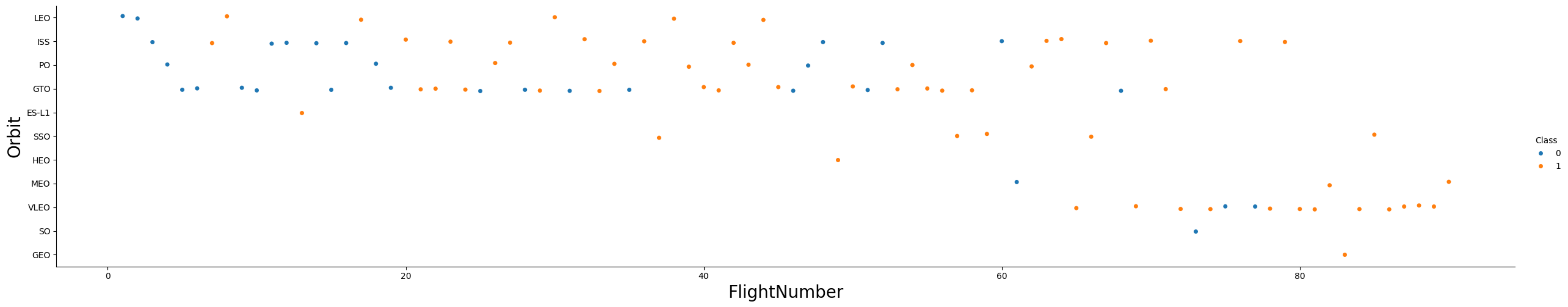
Patterns Observed:
Early flights (low Flight Numbers):Mostly target LEO, ISS, GTO, and PO orbits.
Higher proportion of failures (blue dots).

Later flights (high Flight Numbers): Expanded to more orbit types (SSO, VLEO, MEO, etc.).Successes (orange dots) are much more frequent.

Orbit-specific trends: Missions to LEO and ISS have a mixed early record but show more successes later. GTO flights had more early failures but improved significantly over time. Newer orbits like SSO and VLEO mostly started during the later, more successful flights.

Overall Trend: As SpaceX gained more experience, success rates improved significantly across all orbit types. Mission complexity increased over time, indicated by targeting a greater diversity of orbits.

Flight Number vs. Orbit Type

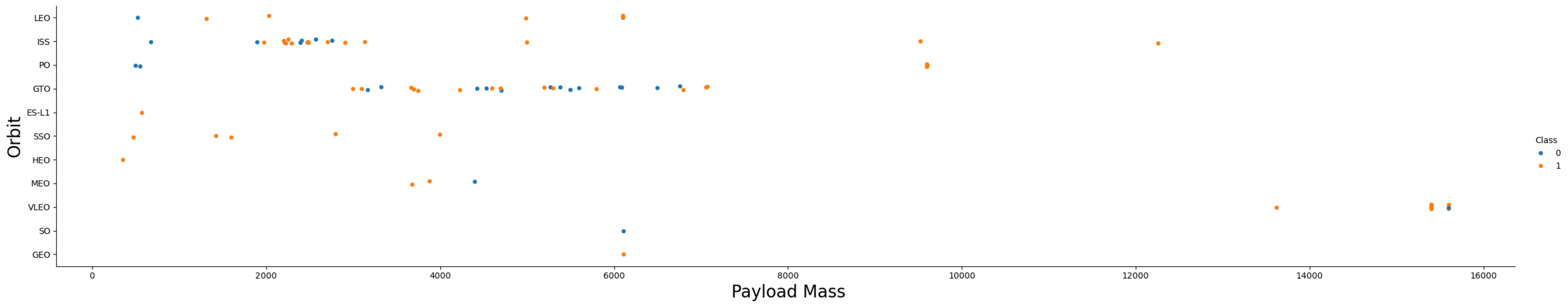


Flight Number vs. Orbit Type

Observations:

1. Early Flights (Flight Numbers < 20–30):
    1. Targeted primarily simpler orbits: LEO, ISS, GTO, PO.
    2. Higher proportion of failures (more blue points).
2. Later Flights (Flight Numbers > 30):
    1. Broader mission types, expanding to orbits like SSO, MEO, GEO.
    2. Much higher success rate (mostly orange points).
3. Orbit-Wise Trends:
    1. LEO and ISS missions initially had mixed success.
    2. GTO missions showed significant early struggles but improved over time.
    3. Newer orbits (SSO, GEO) were introduced during more experienced later missions and mostly succeeded.
4. General Trend:
    1. With experience, SpaceX expanded to more complex orbits and achieved higher success rates across all mission types.

Payload vs. Orbit Type

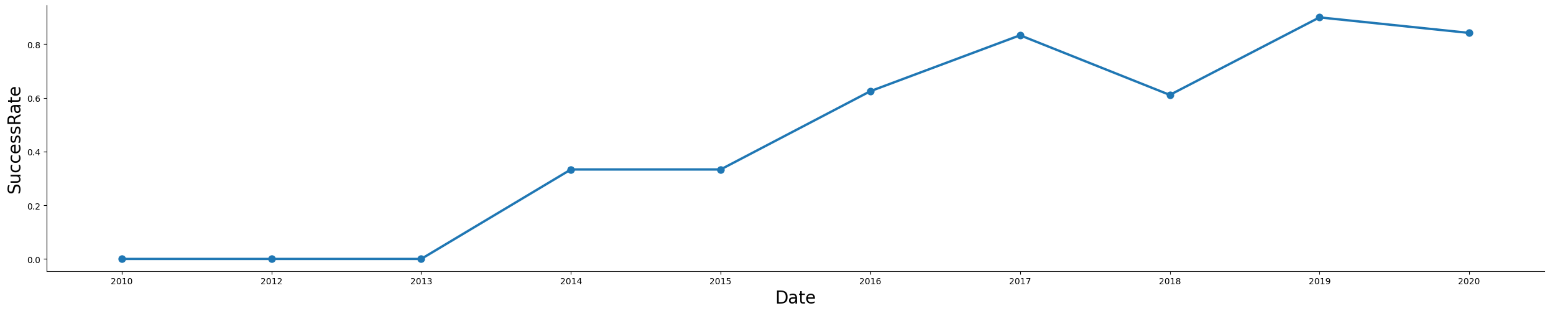


Payload vs. Orbit Type

Observations:

1. Payload Range:
    1. Most payloads are between 2,000 kg and 6,000 kg.
    2. A few missions carried very heavy payloads, up to 16,000 kg.
2. Orbit-specific Patterns:
    1. LEO and ISS missions usually involve smaller to moderate payloads (mostly < 6000 kg).
    2. GTO missions cover a wide range — many heavy payloads (>6000 kg) targeted GTO.
    3. SSO, MEO, and HEO typically had lighter payloads (<4000 kg).
3. Success vs Failure:
    1. Across most orbit types and payload sizes, success (orange points) is more frequent than failure.
    2. Heavier payloads (above 10,000 kg) almost always succeeded.
    3. Failures are more scattered among lighter payloads (<6000 kg).
4. General Trend:
    1. Higher payload masses were associated with higher success rates.
    2. Lighter payload missions showed more variability (some succeeded, some failed).

Launch Success Yearly Trend



Launch Success Yearly Trend

Observations:

1. Early Years (2009–2013):
    1. Success rate was 0% — very few or no successful launches.
2. First Improvement (2014):
    1. Noticeable jump to around 33% success rate.
    2. Suggests early technology and processes were stabilizing.
3. Steady Growth (2015–2017):
    1. Gradual increase in success rate, peaking around 85% by 2017.
4. Temporary Dip (2018):
    1. A small decline in 2018, likely due to a few failed launches.
5. Peak Performance (2019):
    1. Success rate reached its highest point — close to 90%.
6. Slight Decline (2020):
    1. Minor decrease but still maintained a very high success rate (~85%).

All Launch Site Names

- Names of the unique launch sites
- %sql SELECT DISTINCT "Launch_Site" FROM SPACEXTBL
- Query result with a short explanation
  - the SpaceX launches in the dataset were conducted from these different locations.
    - Launch_Site
    - CCAFS LC-40
    - VAFB SLC-4E
    - KSC LC-39A
    - CCAFS SLC-40

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with `CCA`

```
%sql SELECT * FROM SPACEXTBL WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

Launch Site Names Begin with 'CCA'

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Observations:

•All launches in this result happened from Cape Canaveral Air Force Station Launch Complex 40 (CCAFS LC-40).

•These missions occurred between 2010 and 2013.

•The customers were SpaceX and NASA.

•While most missions were successful, the early landing attempts either failed (using parachutes) or did not attempt landing at all.

Total Payload Mass

- Calculate the total payload carried by boosters from NASA
- Present your query result with a short explanation here

```
%sql SELECT SUM("PAYLOAD_MASS__KG_") AS Total_Payload_Mass \
FROM SPACEXTBL \
WHERE "Customer" = 'NASA (CRS)';
```

Total_Payload_Mass
45596

The average payload mass carried by boosters from NASA was approximately 45596 kg

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1
- Present your query result with a short explanation here

Display average payload mass carried by booster version F9 v1.1

In [25]:

```
%sql SELECT AVG("PAYLOAD_MASS__KG_") AS Total_Payload_Mass_booster \
FROM SPACEXTBL \
WHERE "Booster_Version" LIKE 'F9 v1.1%';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[25]:

Total_Payload_Mass_booster
2534.6666666666665

The average payload mass carried by the F9 v1.1 booster version was approximately 2535 kg

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad
- Present your query result with a short explanation here

In [26]:
```
%sql SELECT MIN("Date") AS first_date \
FROM SPACEXTBL \
WHERE "Mission_Outcome" ='Success';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[26]:

first_date
2010-06-04

The first successful SpaceX mission recorded in the dataset took place on June 4, 2010.

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000
- Present your query result with a short explanation here

In [27]:

```
%sql SELECT "Booster_Version" \
FROM SPACEXTBL \
WHERE "Landing_Outcome" = 'Success (drone ship)' \
  AND "PAYLOAD_MASS__KG_" > 4000 \
  AND "PAYLOAD_MASS__KG_" < 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Out[27]:

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

These boosters not only completed their missions successfully but also handled moderate-heavy payloads and achieved drone ship landings — which is technically more challenging than land landings.

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes
- Present your query result with a short explanation here

```
In [37]:
%sql \
SELECT "Mission_Outcome", COUNT(*) AS total \
FROM SPACEXTBL \
GROUP BY "Mission_Outcome";

 * sqlite:///my_data1.db
Done.
Out[37]:
```

Mission_Outcome	total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Most missions (98) were successful, with only a few failures or unclear payload outcomes, although minor data duplication in the "Success" label is seen.

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass
- Present your query result with a short explanation here

In [39]:
```
%sql \
SELECT "Booster_Version", "PAYLOAD_MASS__KG_" \
FROM SPACEXTBL \
WHERE "PAYLOAD_MASS__KG_" = ( \
    SELECT MAX("PAYLOAD_MASS__KG_") \
    FROM SPACEXTBL \
);
```

```
 * sqlite:///my_data1.db
Done.
```

Out[39]:

Booster_Version	PAYLOAD_MASS__KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

- The maximum payload mass carried was 15,600 kg.
- Multiple booster versions (F9 B5 series boosters like B1048.4, B1049.4, etc.) carried this maximum payload.
- All listed boosters successfully handled the heaviest payloads recorded in the dataset.

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- Present your query result with a short explanation here

In [40]:

```
%sql \
SELECT  \
    substr("Date", 6, 2) AS Month, \
    "Landing_Outcome", \
    "Booster_Version", \
    "Launch_Site" \
FROM SPACEXTBL \
WHERE "Landing_Outcome" LIKE 'Failure (drone ship)' \
  AND substr("Date", 1, 4) = '2015';
```

```
 * sqlite:///my_data1.db
Done.
```

Out[40]:

Month	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

•In January 2015 and April 2015, two booster versions (F9 v1.1 B1012 and F9 v1.1 B1015) attempted landings on a drone ship but failed.
•Both failures occurred at the CCAFS LC-40 launch site.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order
- Present your query result with a short explanation here

```
%sql \
SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count \
FROM SPACEXTBL \
WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' \
GROUP BY "Landing_Outcome" \
ORDER BY Outcome_Count DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

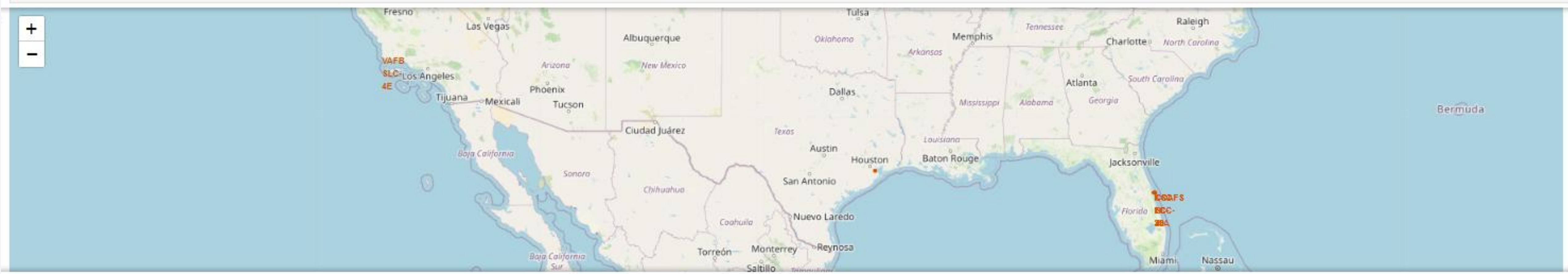
Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

•Most launches (10) did not even attempt landing.
•Drone ship landings were tried the most after that — with 5 successes and 5 failures.
•Ground pad successes and controlled ocean landings were fewer.
•A few landings either failed or were precluded (abandoned drone ship landing). by parachute

Section 3

Launch Sites Proximities Analysis

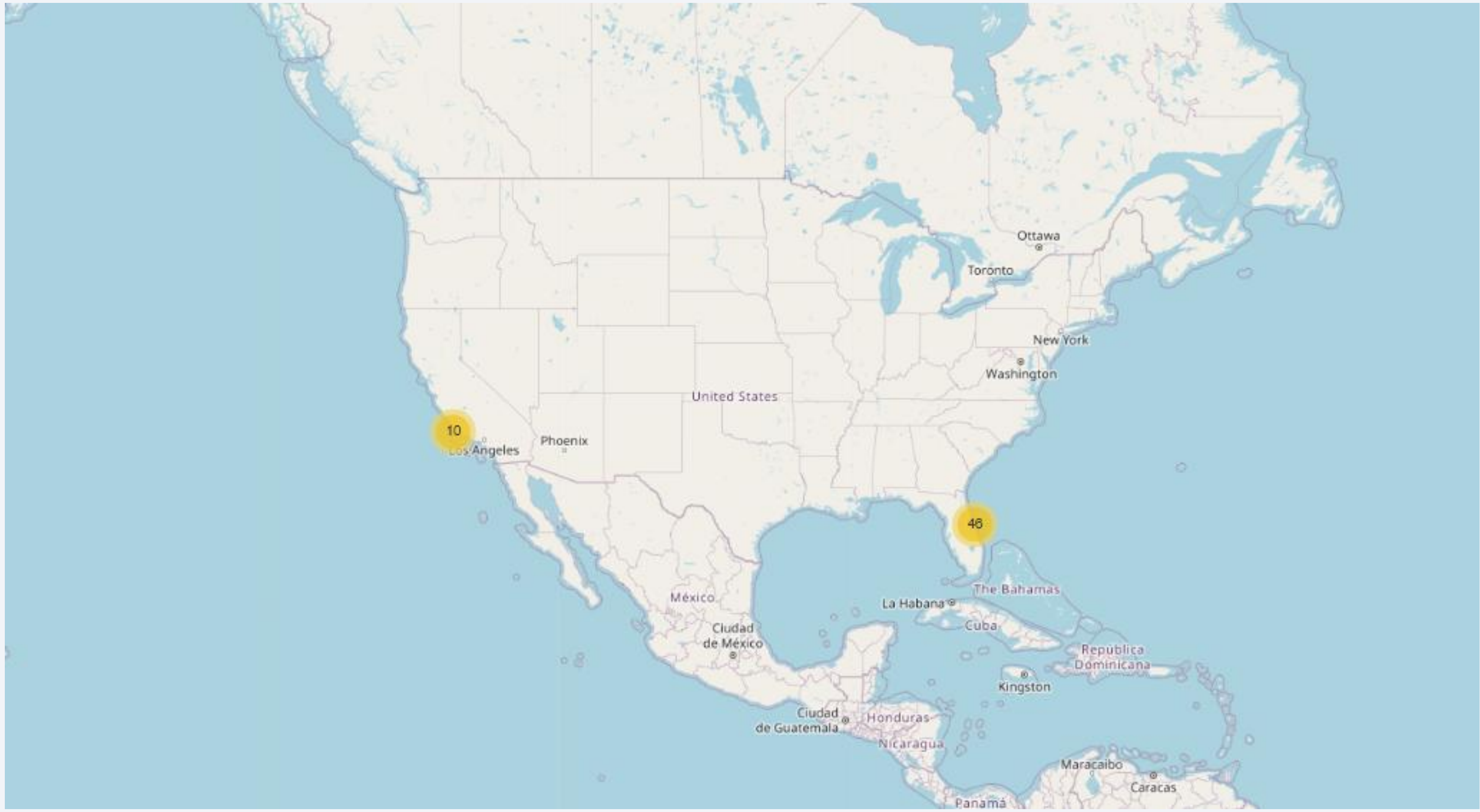
Launch sites' location



Launch sites' location

Markers:
•Each orange label marks a SpaceX launch site.
•Launch Sites Identified:
•On the West Coast (California):
  •VAFB SLC-4E: Vandenberg Air Force Base Space Launch Complex 4E (used for polar and sun-synchronous launches).
•On the East Coast (Florida):
  •CCAFS LC-40
  •KSC LC-39A
  •Others landing zones are also close by.
•Geographical Spread:
•SpaceX uses both coasts

Launch outcomes

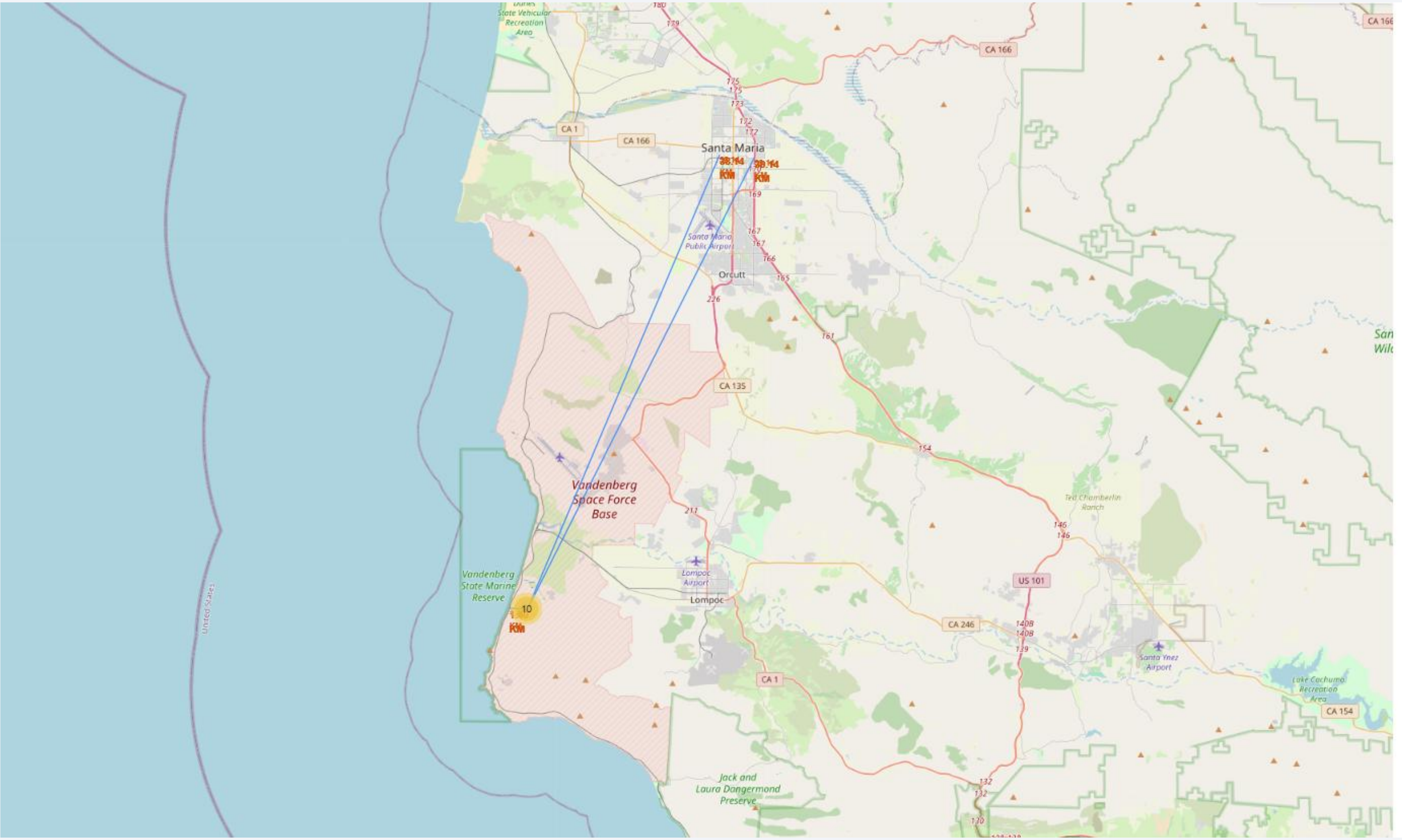


Launch outcomes

Outcomes in the Map:

1. California Cluster (West Coast):
    - 10 launches are clustered here.
    - These launches are mostly for polar orbits or sun-synchronous orbits.
2. Florida Cluster (East Coast):
    - 46 launches are clustered here.

Nearby facilities distance



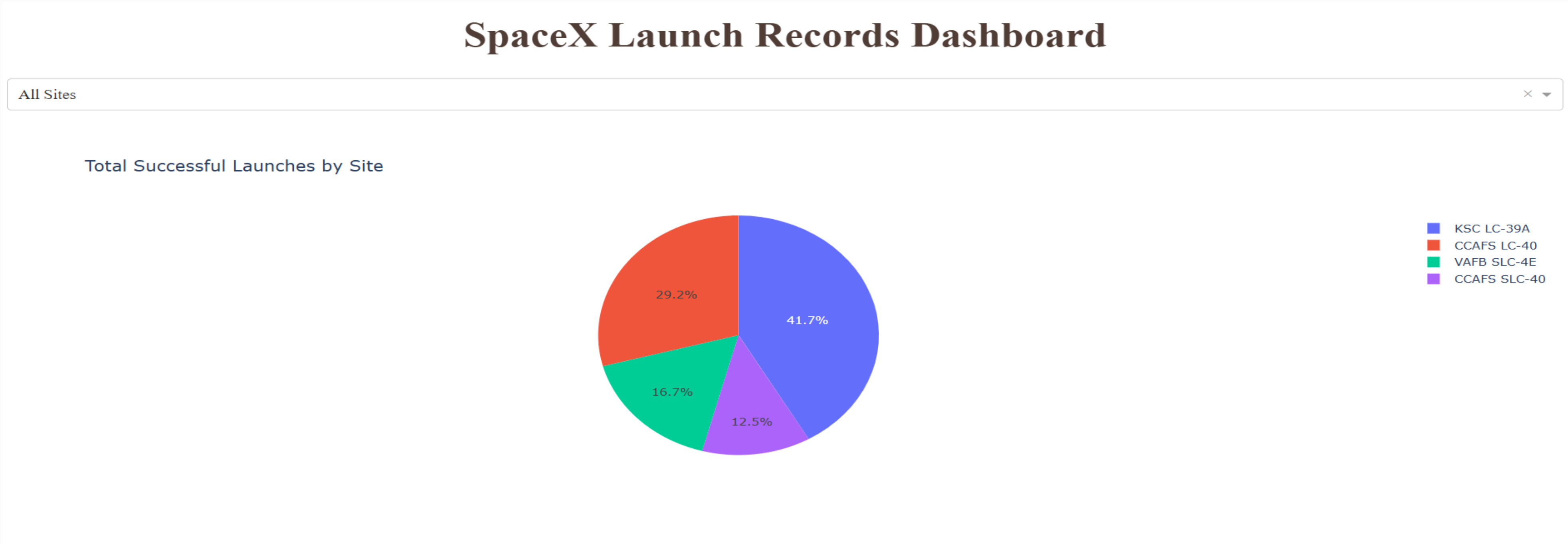
Nearby facilities distance

- The distance lines shows the distance from the railroad, a city and a US 101 which is close to the same city.
- The launch site has close proximity to the coastline, railroad, however, less proximity to cities.

Section 4

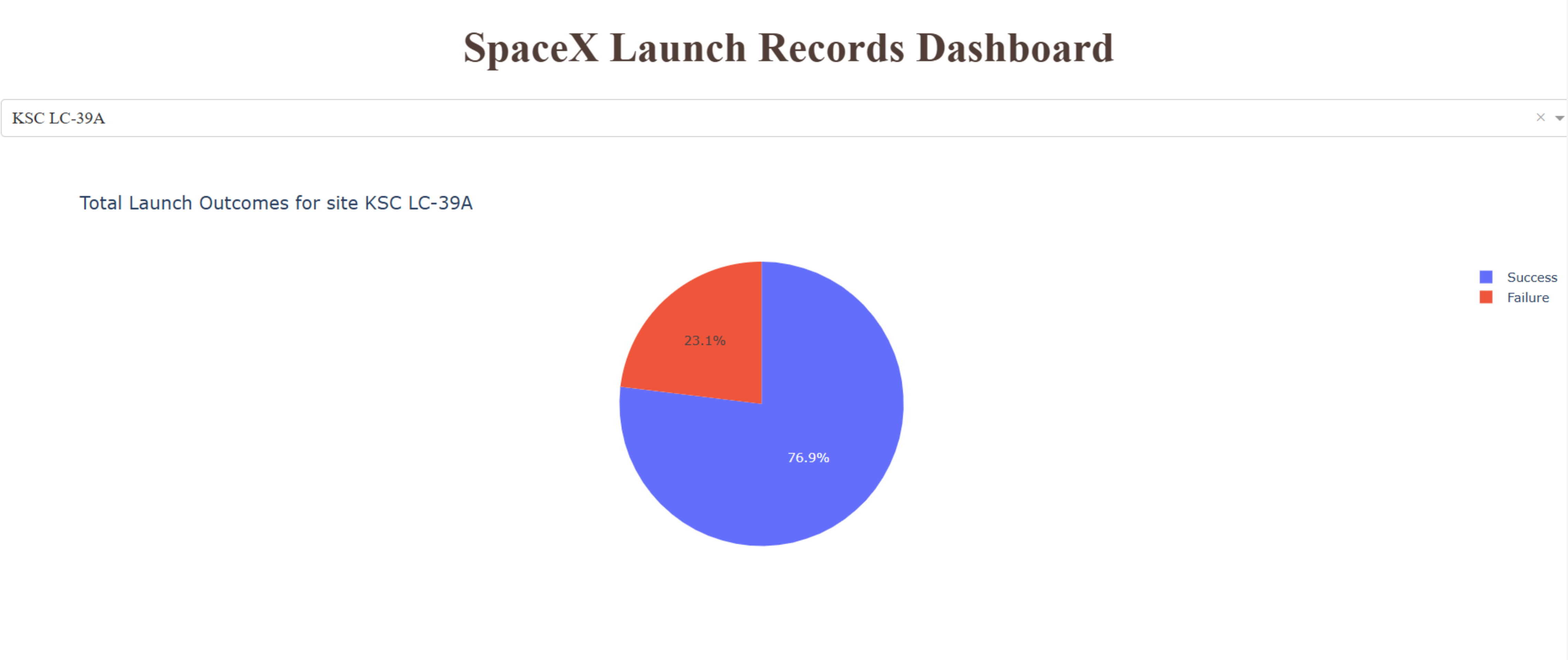
Build a Dashboard with Plotly Dash

All sites: total success count



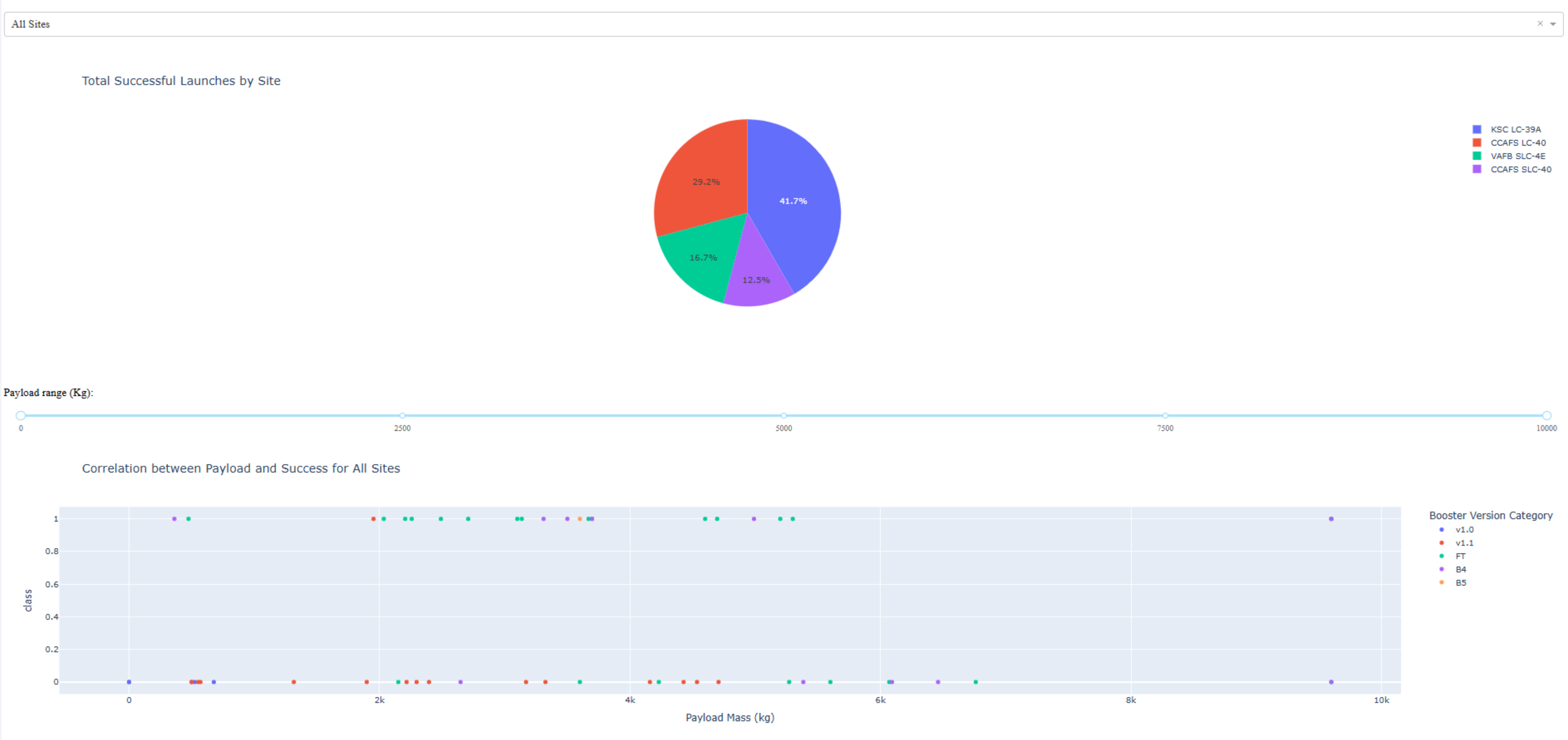
Most successful SpaceX launches happened from KSC LC-39A, followed by CCAFS LC-40, with fewer but notable launches from VAFB SLC-4E.

Launch site with highest launch success ratio



- Most launches (around 77%) from KSC LC-39A were successful.
- About 23% of the launches resulted in failure.
- This indicates a high success rate but still a notable failure rate compared to today's SpaceX standards (where failure is very rare).

Payload vs. Launch Outcome scatter plot for all sites

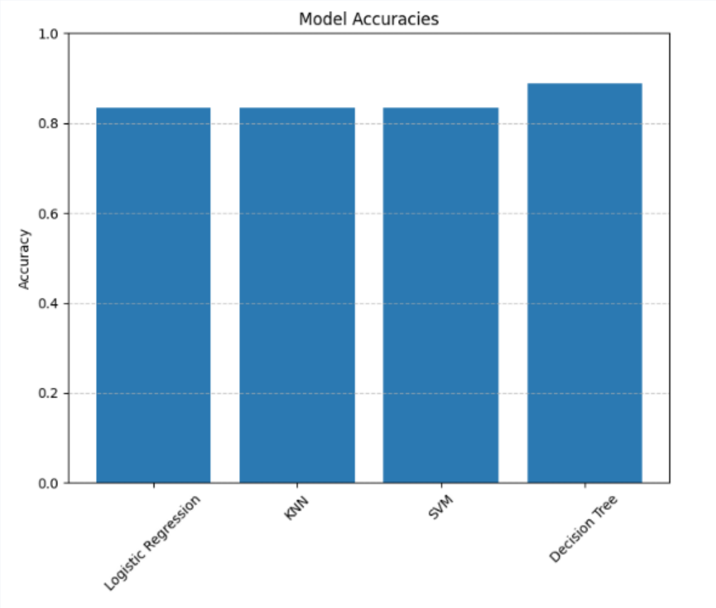


As payload mass increases, the number of launches naturally decreases, but **success rates remain high**, especially for launches using **newer booster versions (B5, FT, B4)**. SpaceX has demonstrated excellent capability in handling mid-to-heavy payloads with remarkable success.

Section 5

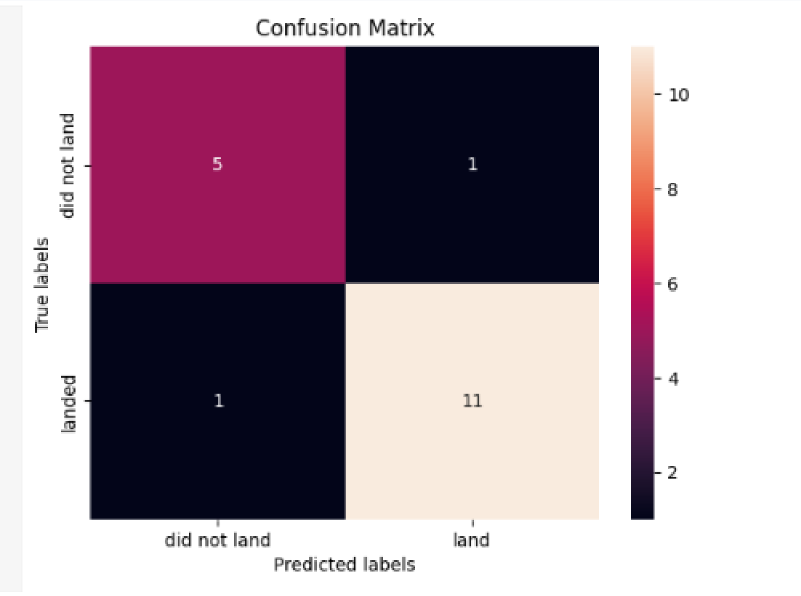
Predictive Analysis (Classification)

Classification Accuracy



Decision tree model has the highest accuracy of 0.88

Confusion Matrix of the Best performing model



Confusion Matrix : interpretation

- 5 (Top-left): 5 rockets correctly predicted as "did not land" (True Negatives).
- 1 (Top-right): 1 rocket was incorrectly predicted as "landed" even though it actually "did not land" (False Positive).
- 1 (Bottom-left): 1 rocket was incorrectly predicted as "did not land" but it actually "landed" (False Negative).
- 11 (Bottom-right): 11 rockets correctly predicted as "landed" (True Positives).

Conclusions

- In this analysis, multiple supervised machine learning models were developed to predict the landing outcome of SpaceX Falcon 9 first stage boosters.
- Models including Logistic Regression, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Decision Tree Classifier were trained and evaluated.
- Performance evaluation through cross-validation and confusion matrices revealed that the models generally achieved high accuracy, with some models performing slightly better after hyperparameter optimization.
- The Decision Tree Classifier, after tuning, emerged as the best-performing model, achieving the highest test accuracy while maintaining a balanced precision and recall for both landing and non-landing predictions.
- The results demonstrate that machine learning models can effectively predict the success of a Falcon 9 first stage landing based on available flight data.
- Further improvements could be made by incorporating more features, additional data, or ensemble methods to boost predictive performance.

Innovative Insights

- In addition to achieving technical objectives of launch cost estimation and booster reuse prediction, our analysis provides actionable strategies for Space Y:
  (1) Proactive booster reuse scheduling,
  (2) Orbit-based pricing,
  (3) Site expansion prioritization,
  (4) Predictive maintenance integration,
  (5) Booster design refinement.

Appendix

	FlightNumber	PayloadMass	Flights	Block	ReusedCount	Orbit_ES-L1	Orbit_GEO	Orbit_GTO	Orbit_HEO	Orbit_ISS	...	Serial_B1058	Serial_B1059	Serial_B1060	Serial_B1062	GridFins_False	GridFins_True	Reused_False	Reused_True	Legs_False	Legs_True
0	1.0	6104.959412	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
1	2.0	525.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
2	3.0	677.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
3	4.0	500.000000	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
4	5.0	3170.000000	1.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
85	86.0	15400.000000	2.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
86	87.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
87	88.0	15400.000000	6.0	5.0	5.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
88	89.0	15400.000000	3.0	5.0	2.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0
89	90.0	3681.000000	1.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	0.0	1.0

Appendix

```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
svm = SVC()
```

```
svm_cv = GridSearchCV(svm, parameters, cv=10)
svm_cv.fit(X_train, Y_train)
```

```
▸ GridSearchCV
  ▸ estimator: SVC
    ▸ SVC
```

```
print("tuned hpyerparameters :(best parameters) ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

```
[22]: parameters = {'criterion': ['gini', 'entropy'],
           'splitter': ['best', 'random'],
           'max_depth': [2*n for n in range(1,10)],
           'max_features': ['auto', 'sqrt'],
           'min_samples_leaf': [1, 2, 4],
           'min_samples_split': [2, 5, 10]}

      tree = DecisionTreeClassifier()
```

```
[23]: tree_cv = GridSearchCV(tree, parameters, cv=10)
      tree_cv.fit(X_train, Y_train)
```

```
▸ GridSearchCV
▸ estimator: DecisionTreeClassifier
  ▸ DecisionTreeClassifier
```

```
print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
tuned hpyerparameters :(best parameters)  {'criterion': 'entropy', 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'splitter': 'best'}
accuracy : 0.8875
```

Thank you!